SIGMIDI Foot Controller v0.1 — Build and bring-up

The firmware targets Arduino Nano ESP32 / ESP32-S3 with 16 MB flash using PlatformIO espressif32 6.9.0 and ESP-IDF 5.3.1. It compiles and passes host tests. Hardware operation, iPad delivery and physical USB flashing require bench validation.

For the complete physical build sequence, use the illustrated assembly guide (printable PDF). It covers part selection, printing, direct Nano mounting, the carrier perfboard, wiring, inspection, calibration and closure.

Build and test

Install PlatformIO Core and a C++17 compiler. ESP-IDF requires paths without spaces; the helper builds in a temporary directory and publishes flashable releases to `distribution/`. Compiler/debug outputs go to disposable `build/`.

```
python3 scripts/build.py
```

Set `PIO` to the PlatformIO executable and `PLATFORMIO_CORE_DIR` to a space-free package cache if needed. On this development machine:

```
PIO=/Users/ab/.platformio/penv/bin/pio PLATFORMIO_CORE_DIR=/tmp/midi-foot-toolchain python3 scripts/build.py
```

Run host tests with `scripts/test.sh`. Set `NODE` to a modern Node executable for browser tests. Set `IDF_PATH` or `PLATFORMIO_CORE_DIR` to installed ESP-IDF sources for cJSON/configuration tests. `python3 scripts/verify_distribution.py` checks the release bundles and complete Fusion archive.

Flash

Run `python3 scripts/serve_flasher.py`, open **http://localhost:8766/** in desktop Chrome or Edge and connect USB-C with a data cable. Choose **Connect & install**. Leave **Erase device** unchecked to keep compatible settings. Before connecting the installer, jumper B1 to GND, tap RST, then remove the jumper and select the new ROM USB serial port. Repeat this for future updates; factory double-reset DFU is not provided. See installer instructions.

For esptool, substitute the actual serial port for `PORT`:

```
python -m esptool --chip esp32s3 --port PORT --baud 460800 write_flash   0x0 distribution/firmware/
  bootloader.bin   0x8000 distribution/firmware/partitions.bin   0x20000 distribution/firmware/firmware.bin
```

The application `.bin` alone requires a compatible bootloader and partition table. Export settings before updating. The controller can clear unusable NVS during startup recovery.

Commissioning

1. Verify the Nano GPIO/harness wiring, mandatory VBUS divider and power rails. Calibrate the ladder and pedals for the assembled hardware. Start with USB, LCD, encoder and ladder. ADC calibration failure disables analog interpretation.
2. Verify the full 280×240 landscape display, RGB order, rounded-corner margins, USB/BLE badge, menus and pedal bars. Select **EDIT CONFIG → ENCLOSURE → LEFT** for the preferred display-on-left assembly or RIGHT for the other enclosure; save.
3. Confirm one menu step per encoder detent in both directions. Default decoding uses four valid transitions; an appropriate half-cycle encoder can use `-DEDGES_PER_DETENT=2`. Hold for 700 ms to cancel/back.
4. Set Modular wiring and **4/6/8/10** switches. Calibrate idle and every installed electrical tap with LCD/BLE active. Validate non-overlapping windows and stable press/release detection. Only one footswitch is supported at a time. Ladder wiring.
5. Connect each TRS MIDI OUT (J3, then J4) to a MIDI receiver with a known-good TRS Type A cable or adapter and verify CC/PC output at 31,250 baud. Check MIDI_OUT_TX idle-high and verify ring (source) / tip

(sink) current-loop polarity on both sockets before connecting valuable equipment. Then configure a CC press message and optional release message. Connect native USB-C to a MIDI host and through the iPad MIDI application's BLE MIDI interface to **SIGMIDI Foot Controller v0.1**. Verify each receiver gets correct channels, values, PC programs, order and delays.

6. Choose Wi-Fi Setup, join the displayed `SIGMIDI-xxxx` network, open **http://192.168.4.1/** and enter the displayed password to acquire editing. Test save/discard, JSON backup, calibration and explicit MIDI Test. Save & Exit disables Wi-Fi and requires stable idle for performance rearming.
7. Test each pedal mode independently. Unplug before changing mode, save and exit, reconnect, then capture endpoints. Verify linear/log response, configured maximum resistance, inversion, output range and stationary behavior.
8. If LEDs are fitted, follow the Pololu 2535 guide. Select RGB / 2535, enable and save. Verify each tap's assignment, white press/yellow release feedback, empty-action behavior and LED-only power cycling.
9. Connect USB and BLE together; compare identical PC/CC values on all three outputs. Repeat on USB-only and VIN/converter-only power to confirm the diode-OR MIDI supply does not back-feed. Check PC+/PC− advances once, not once per transport; check one-link loss does not reset Toggle state while another remains ready. Test disconnect/reconnect, power cycling, setup entry/exit and queue rejection. Measure latency and noise under the intended switch, pedal and display load. Test sustained power/temperature and printed fit on the assembled enclosure.

Display and mechanical installation

The Waveshare 24382 uses four 16.1 mm hollow supports with 2 mm walls. Factory rear M2 studs rest on the supports; fit **M2×5 countersunk screws from below** through Ø2.6 bores. Head-seat Z is −9.51 mm with Ø4.4 access. Nominal stud engagement is 1.39 mm. The factory holes are blind; verify screw engagement and seating without bottoming out.

Base screws are M2×6. The direct Nano mounts use four M1.6×5 PCB screws into integral base posts, with components facing up. Remove rear vent films and open LED covers only for fitted indicators. Confirm the 9 V center-negative inlet and regulated 5 V output before connecting electronics. Provide 3 mm air space under base vents. CAD/assembly guides.

Operating limits

The BLE transport has no persistent bonding. PC numbering is 0–127. Resistor pedals use the nominal 3.3 V / 10 kΩ conversion model. The uppercase LCD font cannot render every character supported by browser names. The firmware supports 32 messages per edge, 128 stored messages total, 16 outstanding switch jobs and up to 10 seconds delay per message. See firmware behavior for configuration and scheduling details.